# 7. 데이터 암호화

# 7. MySQL 데이터 암호화

데이터베이스에는 개인정보나 결제 정보처럼 유출되면 안 되는 데이터가 쌓인다. 서버 접근 권한을 막는 것만으로는 부족하다. 디스크나 백업 파일이 통째로 유출되는 경우도 있기 때문이다. MySQL은 이런 상황에 대비해 데이터를 디스크에 저장할 때 암호화하는 기능을 제공한다.

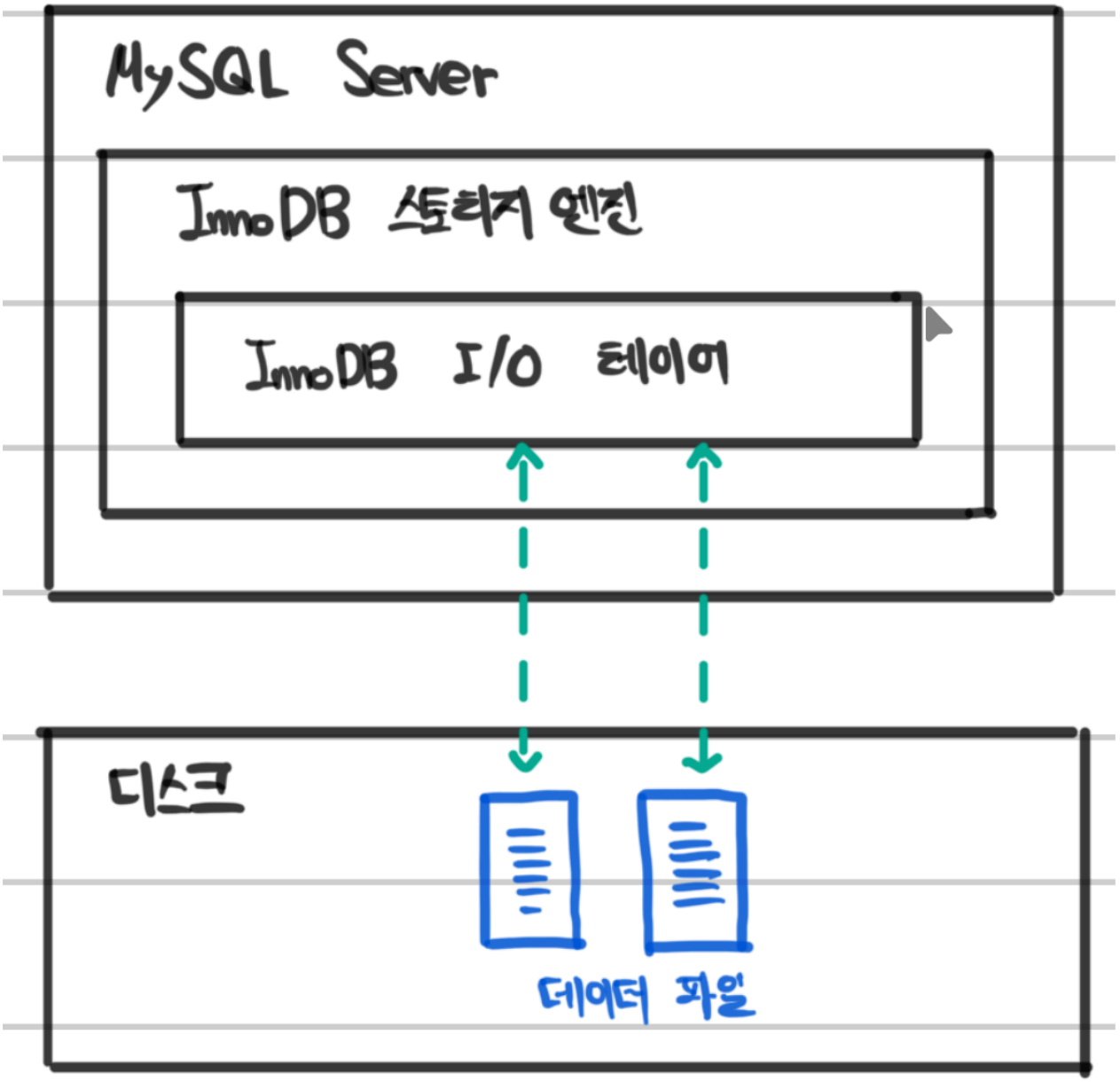
- **MySQL 5.7** : 데이터 파일(테이블스페이스)에 대해서만 암호화 기능을 제공한다.
- **MySQL 8.0** : 데이터 파일뿐만 아니라 Redo Log, Undo Log, 복제를 위한 바이너리 로그까지 암호화를 지원한다.

중요한 정보를 저장하는 서비스에서는 응용 프로그램에서 암호화한 데이터를 DB 서버에서 다시 암호화하는 **이중 암호화**를 선택하기도 한다.

- **응용 프로그램의 암호화** : 주로 중요 정보를 가진 **컬럼 단위**로 암호화를 수행한다.
- **DB 수준의 암호화** : **테이블 단위**로 암호화를 적용한다.

---

## 7.1 MySQL 서버의 암호화




MySQL 서버의 암호화 기능은 **DB 서버와 디스크 사이에서 데이터를 읽고 쓰는 시점에 암호화 혹은 복호화를 수행한다.** 즉, 디스크 I/O 레이어에서만 데이터 암호화·복호화 과정이 실행된다.

그래서 서버가 사용자 쿼리를 처리하는 과정에서는 데이터가 암호화되어 있는지 여부를 식별할 필요가 없다. 암호화된 테이블도 그렇지 않은 테이블과 동일한 처리 과정을 거친다.

데이터 암호화 기능이 활성화되어 있더라도 MySQL 내부와 사용자 입장에서는 아무런 차이가 없다. 그래서 이런 암호화 방식
을 **TDE(Transparent Data Encryption)** 라고 부른다. 여기서

Transparent(투명한)는 "사용자 눈에 보이지 않는다"는 뜻이다. 즉, 데이터는 **디스크에 저장된 단계에서만** 암호화된다.

> TDE는 **저장된 데이터(Data at Rest)** 를 보호한다. 디스크나 데이터 파일, 백업 파일이 유출되어도 키가 없으면 읽을 수 없다. 반면 네트워크로 오가는 데이터는 TDE가 아니라 SSL/TLS로 보호해야 한다.

### 7.1.1 2단계 키 관리

MySQL 서버의 TDE에서 암호화 키는 **KeyRing 플러그인**이 관리하며, 지원되는 KeyRing 플러그인은 다음과 같다.

| 플러그인 | 설명 | 에디션 |
|---|---|---|
| `keyring_file` | 마스터 키를 디스크 파일로 관리 | 커뮤니티 |
| `keyring_encrypted_file` | 마스터 키를 암호화된 파일로 관리 | 엔터프라이즈 |
| `keyring_okv` | Oracle Key Vault 같은 KMIP 호환 키 관리 솔루션 사용 | 엔터프라이즈 |
| `keyring_aws` | AWS KMS(Key Management Service) 사용 | 엔터프라이즈 |

`keyring_file`은 커뮤니티 버전에서 지원하고, 나머지는 엔터프라이즈 버전에서 사용할 수 있다. 키를 관리하는 방법만 다를 뿐 내부적으로 동작하는 방식은 동일하다.

MySQL 서버의 데이터 암호화는 두 가지 종류의 키를 가진다.

- **마스터 키(Master Key)**
- **테이블스페이스 키(Tablespace Key)** : 프라이빗 키(Private Key)라고도 부른다. 공개키 암호화의 개인키와는 관계없고, 외부에 노출되지 않는 키라는 의미다.

MySQL 서버는 외부 키 관리 솔루션 또는 디스크 파일에서 마스터 키를 가져온다. 그리고 암호화된 테이블이 생성될 때마다 해당 테이블을 위한 테이블스페이스 키를 발급한다. MySQL 서버는 마스터 키로 테이블스페이스 키를 암호화해서 각 테이블의 데이터 파일 헤더에 저장한다. 이렇게 생성된 테이블스페이스 키는 테이블이 삭제되지 않는 이상 절대로 변경되지 않는다.

**테이블스페이스 키**

- MySQL 서버 외부로 노출되지 않기 때문에, 주기적으로 변경하지 않아도 보안상 취약점이 되지 않는다.

**마스터 키**

- 외부 파일이나 솔루션에 저장되기 때문에 노출될 가능성이 있어 주기적으로 변경해야 한다.

```
ALTER INSTANCE ROTATE INNODB MASTER KEY;
```

> **마스터 키를 바꾸는 비용이 작은 이유** : 마스터 키를 변경하면 MySQL 서버는 새 마스터 키로 각 테이블스페이스 키만 다시 암호화해서 데이터 파일 헤더에 저장한다. 실제 데이터 파일은 테이블스페이스 키로 암호화되어 있고 이 키는 그대로이므로, 데이터 파일 전체를 다시 암호화할 필요가 없다. 이것이 키를 2단계로 나눈 이유다.

MySQL 서버의 TDE에서 지원하는 암호화 알고리즘은 **AES 256비트**이며, 이외의 알고리즘은 지원하지 않는다.

- 테이블스페이스 키는 `AES-256 ECB(Electronic CodeBook)`로 암호화된다.

- 실제 데이터 파일은 `AES-256 CBC(Cipher Block Chaining)`로 암호화된다.

```
Master Key --(1)--> Tablespace Key --(2)--> InnoDB Data Page

(1) : AES-256 ECB
(2) : AES-256 CBC
```

ECB는 같은 평문 블록이 항상 같은 암호문 블록이 되기 때문에, 반복되는 패턴이 많은 데이터에는 적합하지 않다. 테이블스페이스 키처럼 짧고 무작위인 값에는 문제가 없다. 반면 데이터 페이지는 반복 패턴이 많아서, 이전 블록의 결과를 다음 블록 암호화에 섞는 CBC를 사용한다.

### 7.1.2 암호화와 성능

MySQL 서버의 암호화는 TDE 방식이기 때문에, 디스크에서 한 번 읽은 페이지는 복호화되어 `InnoDB Buffer Pool`에 적재된다. 그래서 한 번 메모리에 적재된 데이터는 암호화되지 않은 테이블과 동일한 성능을 보인다.

하지만 버퍼 풀에 존재하지 않는 데이터 페이지를 읽어야 하는 경우에는 복호화 과정을 거치므로 쿼리 처리가 지연될 수 있다. 또 테이블이 변경되면 변경된 페이지가 디스크로 동기화될 때 암호화되어야 해서 시간이 더 걸린다. 다만 **데이터 페이지를 디스크에 저장하는 작업은 쿼리를 처리하는 스레드가 아니라 MySQL 서버의 백그라운드 스레드가 수행**하므로, 실제 사용자의 쿼리가 지연되는 것은 아니다.

> 새롭게 디스크에서 읽어야 하는 데이터 페이지의 개수만큼 복호화 지연이 발생한다. 버퍼 풀이 충분히 커서 대부분의 데이터가 메모리에 있다면, TDE로 인한 성능 차이는 거의 느껴지지 않는다.

AES는 **16바이트 블록 단위**로 암호화하는 블록 암호다. 평문이 블록 크기보다 짧거나 딱 맞아떨어지지 않으면 패딩이 붙어 암호문이 평문보다 커질 수 있다. 하지만 데이터 페이지(기본 16KB)는 블록 크기보다 훨씬 크고 16바이트의 배수이기 때문에, **암호화 결과가 평문과 동일한 크기의 암호문이 된다.** 즉, TDE를 적용해도 데이터 페이지의 크기가 같으므로 버퍼 풀 효율이나 메모리 사용 효율이 떨어지지 않는다.

#### 7.1.2.1 같은 테이블에 압축과 암호화가 모두 적용되어 있다면?

MySQL 서버는 **압축을 먼저 실행하고, 그다음 암호화를 적용한다.**

일반적으로 암호화된 결과는 매우 무작위한 바이트 배열이 되는데, 이는 압축률을 상당히 떨어뜨린다. 압축은 데이터의 반복 패턴을 찾아 줄이는 방식이라, 패턴이 없는 암호문은 거의 줄어들지 않기 때문이다. 그래서 압축으로 용량을 최소화한 뒤 암호화를 적용한다.

또한 압축된 데이터 페이지는 압축된 상태와 압축 해제된 상태 모두로 InnoDB 버퍼 풀에 존재할 수 있다. 만약 암호화를 먼저 하고 압축을 적용한다면, 버퍼 풀에 있는 압축된 페이지도 암호화된 상태가 되어 사용할 때마다 매번 암호화/복호화 작업을 수행해야 한다.

### 7.1.3 암호화와 복제

레플리카 서버는 소스 서버의 모든 사용자 데이터를 동기화하기 때문에, 실제 데이터 파일도 동일할 것이라고 생각할 수 있다. **하지만 TDE를 사용하면 마스터 키와 테이블스페이스 키는 서로 다르다.**

로컬 디렉토리에서 마스터 키를 관리하는 경우 소스 서버와 레플리카 서버는 당연히 서로 다른 키를 갖게 된다. 키 관리 솔루션을 사용하는 경우에도 소스 서버와 레플리카 서버가 서로 다른 마스터 키를 갖도록 설정해야 한다. **로컬이든 솔루션이든, 소스 서버와 레플리카 서버는 서로 다른 마스터 키를 사용해야 한다.**

복제 소스 서버의 마스터 키를 변경할 때는 `ALTER INSTANCE ROTATE INNODB MASTER KEY` 명령을 실행한다. 이때 명령 자체는 레플리카 서버로 복제되지만, 소스 서버의 마스터 키가 레플리카 서버로 전달되는 것은 아니다. **그래서 마스터 키 로테이션을 실행하면 소스 서버와 레플리카 서버가 각각 서로 다른 마스터 키를 새로 발급받는다.**

> **키링 파일 백업을 잊지 말자.** 데이터 파일만 백업하고 키링 파일을 백업하지 않는 경우가 있는데, 키링 파일을 찾지 못하면 데이터를 복구할 수 없다. 특히 백업 이후에 마스터 키를 변경했다면, 그 백업을 복구하려면 **백업 당시의 마스터 키**가 필요하다. 따라서 TDE의 마스터 키가 언제 변경됐는지 기억해두고, 백업할 때마다 키링 파일도 함께 백업해야 한다.

---

## 7.2 keyring_file 플러그인 설치

| 에디션 | 사용 가능한 플러그인 |
|---|---|
| 엔터프라이즈 에디션 | 다양한 플러그인 사용 가능 |
| 커뮤니티 에디션 | `keyring_file` 플러그인만 사용 가능 |

`keyring_file` 플러그인은 테이블스페이스 키를 암호화하기 위한 마스터 키를 디스크의 파일로 관리한다. **이때 마스터 키는 평문으로 디스크에 저장된다.** 즉, 이 파일이 외부에 노출된다면 데이터 암호화는 무용지물이 된다. 그래서 키링 파일은 데이터 디렉토리와 분리된 경로에 두고, MySQL 서버 계정만 접근할 수 있도록 권한을 최소화해야 한다.

TDE 플러그인은 다른 플러그인과 달리 **MySQL 서버가 시작되는 단계에서 가장 먼저 초기화**되어야 한다. 암호화된 테이블스페이스를 열려면 키가 먼저 준비되어 있어야 하기 때문이다. 그래서 설정 파일에 다음과 같이 명시한다.

- `early-plugin-load` 시스템 변수에 `keyring_file` 플러그인 라이브러리를 명시한다.
- `keyring_file_data` 시스템 변수에 마스터 키를 저장할 파일 경로를 명시한다. 이 경로는 **오직 하나의 MySQL 서버만** 참조해야 한다.

```
[mysqld]
early-plugin-load = keyring_file.so
keyring_file_data =
/very/secure/directory/tde_master.key
```

설정을 마치고 MySQL 서버를 재시작하면 자동으로 `keyring_file` 플러그인이 초기화된다. 초기화 여부는 `SHOW PLUGINS` 명령으로 확인할 수 있다.

```
SHOW PLUGINS;

-- 또는 keyring 플러그인만 확인
SELECT plugin_name, plugin_status
FROM information_schema.plugins
WHERE plugin_name LIKE 'keyring%';
```

> **참고** : MySQL 8.0.24부터는 플러그인을 대체하는 **키링 컴포넌트** (`component_keyring_file` 등)가 도입되었고, `keyring_file` 플러그인은 이후 deprecated되어 MySQL 8.4에서 제거되었다. 새로 구축한다면 사용하는 버전에 맞춰 컴포넌트 방식을 확인하는 것이 좋다.

---

## 7.3 테이블 암호화

### 7.3.1 테이블 생성

TDE를 이용하는 테이블은 다음과 같이 생성할 수 있다.

```
CREATE TABLE tab_encrypted (
    id INT,
    data VARCHAR(100),
    PRIMARY KEY(id)
) ENCRYPTION='Y';
```

일반적인 테이블 생성 구문과 동일하며, 마지막에 `ENCRYPTION='Y'` 옵션만 추가하면 된다.

이 테이블의 데이터가 디스크에 기록될 때는 자동으로 암호화되어 저장되고, 다시 디스크에서 메모리로 읽어올 때 복호화된다. 테이블의 암호화 여부는 `information_schema`의 `TABLES` 뷰에서 `CREATE_OPTIONS` 컬럼으로 확인할 수 있다.

```
-- 특정 테이블의 암호화 여부 확인
SELECT table_schema, table_name, create_options
FROM information_schema.tables
WHERE table_name = 'tab_encrypted';

-- 암호화된 테이블만 검색
SELECT table_schema, table_name
FROM information_schema.tables
WHERE create_options LIKE '%ENCRYPTION=''Y''%';
```

이미 만들어진 테이블도 `ALTER TABLE`로 암호화를 적용하거나 해제할 수 있다. 이때 테이블 데이터가 새로 작성되므로, 큰 테이블이라면 시간이 오래 걸린다.

```
ALTER TABLE tab_plain ENCRYPTION='Y';
```

하지만 테이블을 생성할 때마다 옵션을 직접 적으면 실수로 암호화를 빠뜨릴 수 있다. MySQL 서버의 모든 테이블에 암호화를 적용하고 싶다면 `default_table_encryption` 시스템 변수를 `ON`으로 설정하면 된다. 그러면 별도로 옵션을 적지 않아도 새로 생성되는 스키마와 테이블에 암호화가 적용된다. (이미 존재하는 테이블은 자동으로 암호화되지 않으므로 직접 `ALTER TABLE`해야 한다)

### 7.3.2 응용 프로그램 암호화와의 비교

응용 프로그램에서 직접 암호화해서 MySQL 서버에 저장하는 경우도 있다. 이 경우 MySQL 서버는 데이터가 암호화되었다는 사실을 인지하지 못한다. 그래서 인덱스를 생성하더라도 인덱스의 기능을 100% 활용할 수 없다.

```
CREATE TABLE app_user (
    id BIGINT,
    enc_birth_year VARCHAR(50), -- 응용 프로그램에서 미리 암호화해서 저장하는 컬럼
    ...
    PRIMARY KEY(id),
    INDEX ix_birthyear(enc_birth_year)
);
```

테이블 자체는 암호화되지 않았지만, `enc_birth_year` 컬럼에는 이미 암호화된 값이 저장되어 있다.

```
-- (1) 동등 비교 : 가능
SELECT * FROM app_user WHERE enc_birth_year = #{encryptedYear};

-- (2) 범위 검색 : 의미 없는 결과
SELECT * FROM app_user WHERE enc_birth_year BETWEEN #{encryptedMinYear} AND #{encryptedMaxYear};
```

- (1)은 같은 값인지만 비교하므로, 검색할 값을 똑같이 암호화해서 비교하면 인덱스를 사용해 검색할 수 있다.
- 하지만 (2)처럼 출생연도 범위로 검색하거나 출생연도 기준으로 정렬하는 쿼리는 사용할 수 없다. 암호화된 값은 원래 값의 크기 순서를 유지하지 않기 때문이다. 1990의 암호문이 2000의 암호문보다 작다는 보장이 없다.

> (1)이 가능하려면 같은 평문이 항상 같은 암호문이 되도록 암호화해야 한다. 하지만 이렇게 하면 같은 값을 가진 행끼리 암호문도 같아서, 값의 분포가 드러나는 약점이 생긴다. 응용 프로그램 암호화는 보안과 검색 편의성 사이의 트레이드오프가 있다.

반면 TDE를 사용하면 인덱스 관련 작업을 모두 처리한 후, 최종적으로 디스크에 데이터 페이지를 저장할 때만 암호화하기 때문에 이 같은 제약이 없다.

이 책에서는 응용 프로그램의 암호화와 MySQL 서버의 암호화 중 하나를 선택해야 한다면 **MySQL 서버의 암호화 기능을 선택할 것을 권장**한다. 하지만 둘은 막아주는 위협이 다르다.

| 구분 | TDE | 응용 프로그램 암호화 |
|---|---|---|
| 막아주는 위협 | 디스크·데이터 파일·백업 파일 유출 | DB 계정 탈취, 내부자의 데이터 조회 |
| DB에 로그인하면 | 모든 데이터를 **평문**으로 확인 가능 | 평문 확인 **불가** |
| 인덱스·검색 | 제약 없음 | 범위 검색, 정렬 불가 |
| 적용 단위 | 테이블 | 컬럼 |

그래서 정말 중요한 정보라면 두 방식을 함께 쓰는 이중 암호화를 선택한다.

### 7.3.3 테이블스페이스 이동

MySQL 서버에는 테이블스페이스 파일만 다른 서버로 복사하는 **Transportable Tablespace** 기능이 있다. TDE가 적용된 테이블은 원본 MySQL 서버와 목적지 MySQL 서버의 마스터 키가 다르기 때문에 주의해야 한다.

먼저 암호화되지 않은 테이블의 이동 과정을 보자. 원본 서버에서 `FLUSH TABLES` 명령으로 테이블스페이스를 export할 수 있다.

```
FLUSH TABLES source_table FOR EXPORT;
```

이 명령이 실행되면 MySQL 서버는 `source_table`의 저장되지 않은 변경사항을 모두 디스크에 기록하고, 더 이상 `source_table`을 변경할 수 없도록 잠금을 건다. 동시에 `source_table`의 구조를 `source_table.cfg` 파일로 기록해둔다.

그러면 `source_table.ibd`와 `source_table.cfg` 파일을 목적지 서버로 복사한다. 복사가 모두 완료되면 `UNLOCK TABLES` 명령을 실행해 `source_table`을 다시 사용할 수 있게 하면 된다.

목적지 서버에서는 같은 구조의 테이블을 만들어 두고, 테이블스페이스를 교체한다.

```
-- 목적지 서버
CREATE TABLE source_table (...);                -- 원본과 같은 구조로 생성
ALTER TABLE source_table DISCARD TABLESPACE;    -- 빈 .ibd 파일 제거
-- 복사한 source_table.ibd, source_table.cfg(.cfp)를 데이터 디렉토리에 배치
ALTER TABLE source_table IMPORT TABLESPACE;     -- 복사한 파일을 테이블스페이스로 등록
```

여기까지가 암호화되지 않은 테이블의 이동 과정이다. TDE가 적용된 테이블을 export하면 다음 과정이 추가로 일어난다.

- MySQL 서버는 임시로 사용할 마스터 키를 발급해서 `source_table.cfp` 파일로 기록한다.
- 암호화된 테이블스페이스 키를 기존 마스터 키로 복호화한 뒤, 임시로 발급한 마스터 키로 다시 암호화해서 데이터 파일의 헤더에 저장한다.

목적지 서버는 `.cfp` 파일의 임시 마스터 키로 테이블스페이스 키를 복호화하고, 자신의 마스터 키로 다시 암호화해서 저장한다.

> **반드시 데이터 파일과 임시 마스터 키가 저장된 `*.cfp` 파일을 함께 복사해야 한다.** `*.cfg` 파일은 단순히 테이블 구조만 가지고 있기 때문에 없어져도 경고만 발생하고 테이블스페이스를 복구할 수 있다. 하지만 `*.cfp` 파일이 없어지면 테이블스페이스 키를 복호화할 방법이 없어 **복구가 불가능해진다.**

---

## 7.4 Undo Log 및 Redo Log 암호화

테이블 암호화를 적용하더라도 **디스크에 저장되는 테이블 데이터만** 암호화된다. MySQL 서버의 메모리에 있는 데이터는 복호화된 평문으로 관리되며, 이 평문 데이터가 테이블의 데이터 파일 이외의 디스크 파일로 기록되는 경우에는 여전히 평문으로 저장된다. 그래서 테이블 암호화를 적용해도 Redo Log, Undo Log, Binary Log에는 평문으로 저장된다.

MySQL 8.0부터는 `innodb_undo_log_encrypt` 시스템 변수와 `innodb_redo_log_encrypt` 시스템 변수를 이용해 InnoDB 스토리지 엔진의 Undo Log와 Redo Log를 암호화된 상태로 저장할 수 있게 개선되었다. 바이너리 로그는 별도로 `binlog_encryption` 시스템 변수로 암호화한다.

```
SET GLOBAL innodb_redo_log_encrypt = ON;
SET GLOBAL innodb_undo_log_encrypt = ON;
```

테이블 암호화는 테이블 하나에 암호화를 적용하면 해당 테이블의 모든 데이터가 암호화된다. 하지만 Redo Log와 Undo Log는 그렇게 적용할 수 없다. 실행 중인 MySQL 서버에서 암호화를 활성화하더라도, 기존의 모든 Redo Log와 Undo Log 데이터를 그 시점에 한 번에 암호화해서 다시 저장할 수는 없다.

- **활성화하면** : 그때부터 생성되는 Redo Log와 Undo Log만 암호화해서 저장한다. 이전 로그는 평문으로 남아 있다.
- **비활성화하면** : 그때부터 저장되는 로그만 평문으로 저장한다. 이미 암호화되어 저장된 로그는 여전히 암호화된 상태로 남는다.

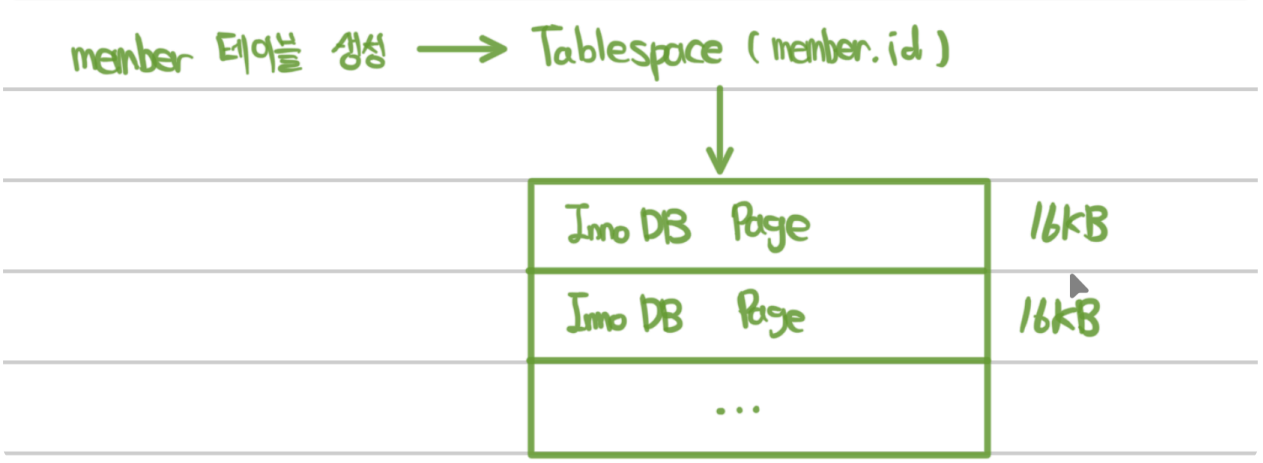
그래서 암호화를 비활성화했더라도, 이전에 암호화된 로그가 남아 있는 동안에는 상황에 따라 며칠 또는 몇 달 동안 **여전히 암호화 키가 필요할 수 있다.** 이 기간에 키링 파일을 지우거나 플러그인을 빼면 서버를 정상적으로 시작하지 못할 수 있다.

---

## 보완

### MySQL의 테이블스페이스

테이블스페이스는 InnoDB가 테이블의 실제 데이터를 디스크에 저장하기 위해 사용하는 **논리적인 저장 공간**이다. 즉, 여러 개의 InnoDB 페이지를 담고 있는 공간이다.




여기서 **페이지(Page)** 는 InnoDB가 디스크 I/O를 수행하는 기본 단위이며, 기본 크기는 16KB다(`innodb_page_size`). 어떤 row 하나만 알고 싶어도, 디스크에서는 그 row가 들어 있는 페이지 단위로 읽어 버퍼 풀에 올린다. TDE의 암호화·복호화도 이 페이지 단위로 일어난다.

### 왜 파일이라고 하지 않고 테이블스페이스라고 할까?

> 테이블스페이스는 **논리적인** 개념이고, 파일은 **물리적인** 개념이다.

```
System Tablespace
|-- ibdata1
|-- ibdata2
|-- ibdata3

하나의 테이블스페이스가 여러 데이터 파일을 가질 수 있다.
```

**테이블스페이스라는 논리적인 저장 공간이 실제로는 하나 이상의 파일에 저장된다.** 위 예시처럼 시스템 테이블스페이스는 여러 개의 데이터 파일로 구성될 수 있다. 반면 File-Per-Table 테이블스페이스는 테이블 하나 당 `.ibd` 파일 하나다.

MySQL InnoDB에는 여러 종류의 테이블스페이스가 있다.

| 종류 | 설명 |
| --- | --- |
| `System Tablespace` | InnoDB 내부 시스템 데이터(체인지 버퍼 등)를 저장한다 (`ibdata1`) |
| `File-Per-Table Tablespace` | 테이블 하나마다 별도의 `.ibd` 파일을 만든다 |
| `General Tablespace` | 여러 테이블을 하나의 테이블스페이스에 저장할 수 있다 |
| `Undo Tablespace` | 언두 로그를 저장한다 |
| `Temporary Tablespace` | 임시 테이블 등의 데이터를 저장한다 |

가장 많이 접하는 것은 `File-Per-Table Tablespace` 다. `innodb_file_per_table` 기본값이 `ON`이기 때문이다. InnoDB에서는 데이터뿐만 아니라 **인덱스도** 같은 테이블스페이스 안에 저장된다.

## 왜 테이블스페이스만 옮기는 기능을 자주 사용할까?

테이블 전체를 SQL로 덤프하고 다시 복원하는 것보다 훨씬 빠르게 물리 데이터를 옮길 수 있기 때문이다. `innodb_file_per_table=ON`이라면 테이블 하나의 데이터와 인덱스가 거의 전부 그 테이블의 `.ibd` 파일에 들어 있다.

### 논리 백업

```
mysqldump → INSERT 문으로 변환 → 다른 서버에서 INSERT 실행
→ 인덱스 다시 구성 → 데이터 재생성
```

### Transportable Tablespace

```
기존 서버 member.ibd → 새 서버로 복사 → IMPORT TABLESPACE
```

- 이미 만들어진 InnoDB 페이지들을 그대로 가져온다.
- 데이터를 다시 생성하는 것이 아니라, 이미 만들어진 물리적 페이지를 옮기는 것이다.

예를 들어 `orders` 테이블만 다른 서버로 옮겨야 한다면, DB 전체를 백업할 필요 없이 `orders.ibd`만 대상으로 할 수 있어 효율적이다.

테이블스페이스 안에는 데이터뿐만 아니라 인덱스 구조도 이미 만들어져 있다. 논리적으로 데이터를 다시 넣으면 InnoDB가 행을 하나씩 삽입하면서 B+Tree를 새로 만들고, 페이지를 분할하고, 인덱스를 구성해야 한다. 하지만 `.ibd` 파일을 그대로 가져오면 이 과정이 이미 끝나 있으므로, 데이터가 클수록 차이가 커진다.

> **주의** : 파일을 그대로 옮기는 방식이라 제약이 있다. 원본과 목적지의 페이지 크기(`innodb_page_size`)가 같아야 하고, 목적지에 같은 구조의 테이블이 미리 있어야 하며, MySQL 버전 간 호환성도 확인해야 한다.